

HANDWRITTEN DIGIT RECOGNITION USING NEURAL NETWORKS AND BACKPROPAGATION

Submitted By:

Bisma Raza

Roll Number:

BSE-F23-M47

Submitted To:

Mr. Asif Raza

Course Title:

Artificial Intelligence



University Of Mianwali

Department Of Software Engineering

Table of Contents

Abstract

I. Introduction

II. Literature Review

III. Methodology

IV. Neural Network Architecture

V. Results and Discussion

5.1 Training Loss Analysis

5.2 Training Accuracy Analysis

5.3 Confusion Matrix Analysis

5.4 Sample Prediction Analysis

5.5 Overall Discussion

VI. Conclusion

VII. Future Work

References

Appendix A: Implementation Summary

Appendix B: Generated Results

Abstract

This project describes a handwritten digit recognition system built with a feed-forward neural network trained using backpropagation. The main goals were to achieve high classification accuracy and to study how neural networks learn. The model was coded in Python and NumPy, without using deep learning libraries. The system uses the scikit-learn digits dataset, where each digit is an 8×8 grayscale image, giving 64 input features. The network has 64 input neurons, one hidden layer with 32 neurons, and 10 output neurons for digits 0 to 9. The hidden layer uses the sigmoid activation function, and the output layer uses softmax. Cross-entropy is the loss function, and gradient descent updates the weights. The final test accuracy was about 94.44% (developers, 2023). Looking at the loss and accuracy curves, confusion matrix, and sample predictions shows that the model learned digit patterns well. This project helped me understand forward propagation, backpropagation, and gradient-based learning in detail.

Keywords: Backpropagation, Neural Network, Handwritten Digit Recognition, Machine Learning, Gradient Descent, NumPy, Classification.

I. Introduction

Artificial neural networks connect layers to map input data to output. For classification, the model takes features and predicts the correct class. In this project, the input is a grayscale image of a digit, and the output is a label from 0 to 9.

Backpropagation helps neural networks learn from mistakes by sending the output error backward to update the weights. I chose this method to clearly show how neural networks get better at making predictions during training.

Handwritten digit recognition is a good example of pattern recognition. Since digits can look very different, the model learns features like curves and strokes instead of fixed shapes. This kind of recognition is useful in real-world tasks like sorting mail, processing cheques, reading forms, and digitizing documents.

The system uses a simple fully connected neural network with a clear workflow: loading data, normalizing inputs, splitting data, encoding labels, running forward propagation, calculating loss, applying backpropagation, and evaluating results. I left out advanced techniques on purpose to keep things clear and focus on learning.

II. Literature Review

Machine learning from examples has a long history. Rosenblatt created the perceptron, an early model that could adjust its weights and make decisions from input data [1]. The perceptron brought

in the idea of weighted inputs and learning from examples. However, a single-layer perceptron cannot solve nonlinear problems, which led to the development of multilayer neural networks.

Backpropagation made it possible to train multilayer neural networks in practice. Rumelhart, Hinton, and Williams showed how errors can be sent backward through hidden layers using the chain rule of calculus [2]. Their work is the main theory behind this project, since the learning process relies on computing gradients and updating weights.

Handwritten and document recognition have also been widely studied. LeCun and colleagues showed that gradient-based learning works well for document recognition tasks [3]. While their approach is more advanced than a basic fully connected model, it supports the idea that neural networks can learn digit patterns directly from pixel data.

Standard machine learning texts explain the same supervised learning workflow used in Standard machine learning books describe the same supervised learning steps used here: choosing data, preprocessing, picking a model, setting a loss function, training, and testing on new data [4], [5]. The tools I used, like scikit-learn for the dataset and evaluation, are explained by Pedregosa et al. [7]. These sources support both the theory and the tools in this project. The t-digits dataset was loaded first. Each sample is an 8×8 grayscale image which, when flattened, becomes a vector of 64 input features. The target label is an integer from 0 to 9.

I normalized the inputs from their original range of 0–16 to a range of 0–1. Labels were one-hot encoded, so if the digit is 3, only the fourth value in a 10-element vector is 1 and the rest are 0. Normalization helps the neural network train better and faster. (Klambauer et al., 2020)

I split the dataset into training and test sets using an 80–20 ratio. Stratified splitting made sure all digit classes were balanced in both sets. The training set was used to update the model, and the test set was kept separate for final evaluation.

The network has three layers. The input layer has 64 neurons for the pixel values. The hidden layer has 32 neurons and uses the sigmoid activation function. The output layer has 10 neurons with softmax activation, giving class probabilities. The model predicts the digit with the highest probability.

I used cross-entropy loss because it works well for multi-class classification. (Categorical Cross-Entropy in Multi-Class Classification, 2025) Training used full-batch gradient descent with a learning rate of 0.5 for 500 epochs, which gave stable and accurate results. I used Python and NumPy for calculations, Matplotlib for graphs, and scikit-learn for data and metrics. No deep learning frameworks were used, so every step of backpropagation is clear.

The network has two weight matrices (input-to-hidden and hidden-to-output) and two bias vectors. I randomly initialized the weights with a fixed seed for reproducibility, and set the biases to zero. In each epoch, the model did a forward pass, calculated cross-entropy loss, ran backpropagation to get gradients, and updated the parameters with gradient descent. I recorded training and testing accuracy each epoch to create learning curves.

IV. Neural Network Architecture

The feedforward neural network I used has three layers. Its structure can be written as $64 \rightarrow 32 \rightarrow 10$, where each number shows how many neurons are in each layer. The table below gives a summary of the full architecture:

Input Layer	64	None	One neuron per pixel feature
Hidden Layer	32	Sigmoid	Feature extraction & pattern learning
Output Layer	10	Softmax	Class probability for digits 0–9

Table 1: Neural Network Architecture Summary

The sigmoid activation function adds non-linearity to the hidden layer, which helps the network learn complex patterns. (Albahli et al., 2020) The softmax function in the output layer turns the activations into a probability distribution over the 10 digit classes. (Handwritten Digit Recognition using Neural Network, 2025) Cross-entropy loss works well with softmax for multi-class problems. (1.17. Neural network models (supervised) — scikit-learn 1.8.0 documentation, 2026)

V. Results and Discussion

The model learned to classify handwritten digit images well. At first, the randomly set weights gave unreliable predictions. As training went on for 500 epochs, the loss went down and accuracy improved, showing that backpropagation helped the model learn correctly.

In the conducted experimIn this experiment, the final test accuracy was about 94.44%. (developers, 2020) This is a good result for a small, fully connected network trained from scratch on low-resolution 8×8 images. It shows that with good preprocessing and correct backpropagation, even a simple model can learn useful digit patterns.

The training loss curve went down steadily over the epochs. Since loss measures prediction error, this steady drop means the gradient descent updates worked well. If the loss had gone up or been uneven, it would have meant problems with the learning rate, gradient formulas, or the code. The smooth drop here shows the model learned as expected.

[Figure 1: Training Loss — Decreases Over 500 Epochs]

Figure 1. Training loss decreases over epochs, confirming successful learning.

5.2 Training Accuracy Analysis

The accuracy graph shows both training and testing accuracy over the epochs. Both improved steadily during training, and the gap between them stayed small. This means the model was able

to generalize to new test data instead of just memorizing the training set, so overfitting was not a big issue here.

[Figure 2: Training & Testing Accuracy Over 500 Epochs]

Figure 2. Training and testing accuracy improve over epochs with a small generalization gap.

5.3 Confusion Matrix Analysis

The confusion matrix shows how well the model did for each digit by comparing its predictions to the actual labels. Strong diagonal values mean correct predictions, while off-diagonal values are mistakes. The matrix had strong diagonals for digits 0, 2, 3, 4, 5, 6, and 7. Most errors happened between digits that look similar, like some 8s being predicted as 1s, and some 1s as 9s. These mistakes make sense because the images are low resolution (8×8).

[Figure 3: Confusion Matrix — 10×10 Digit Classification]

Figure 3. Confusion matrix for digit classification showing strong diagonal values.

5.4 Sample Prediction Analysis

I generated sample predictions from test data to make the results easier to understand. Labels are shown as T:<actual> P:<predicted>. Most examples matched, like T:5 P:5 and T:2 P:2. One mismatch, T:1 P:6, shows where the model still has trouble. Looking at these examples helps explain the types of mistakes the model makes.

[Figure 4: Sample Handwritten Digit Predictions]

Figure 4. Sample predictions showing true (T) and predicted (P) labels.

5.5 Overall Discussion

The results show that a well-implemented backpropagation neural network can reach high accuracy in multi-class digit classification. One main limitation is that this model treats each image as a flat vector and ignores spatial structure. Convolutional neural networks (CNNs) work better for images because they can find local patterns like edges and curves using shared filters (Kociolek et al., 2022). Still, for showing backpropagation from the basics, the fully connected approach is clear and useful for learning.

VI. Conclusion

In this project, I built a full backpropagation neural network for handwritten digit recognition from scratch using Python and NumPy. The implementation covered forward propagation, cross-

entropy loss calculation, backpropagation of gradients, weight updates with gradient descent, making predictions, and showing results.

The model reached about 94.44% test accuracy on the scikit-learn digits dataset. The loss curve, accuracy curve, confusion matrix, and sample predictions all show that the model learned useful features from the digit images. I did all calculations by hand instead of using a deep learning framework, which helped me understand backpropagation in a practical way.

These results show that backpropagation is a practical and effective way to train multilayer neural networks. Even with a small network and a simple dataset, accurate digit classification is possible if preprocessing, training, and evaluation are done well.

VII. Future Work

Several extensions would improve this system:

- Replace the fully connected network with a Convolutional Neural Network (CNN) for better spatial feature extraction from image data.
- Implement mini-batch gradient descent instead of full-batch training to scale to larger datasets more efficiently.
- Experiment with ReLU activation in the hidden layer, which often trains faster and mitigates the vanishing gradient problem compared to sigmoid.
- Add regularization techniques such as dropout or L2 weight decay to reduce overfitting on larger, more complex datasets.
- Benchmark the model on the full MNIST dataset (70,000 samples) to evaluate scalability.
- Implement weight serialization to save and reload trained models without retraining.
- Develop an interactive interface allowing users to draw a digit with a mouse and obtain a real-time model prediction.

These improvements would make the system more useful in practice while still keeping its value as a learning tool built from the basics.

References

- [1]F. Rosenblatt, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958. DOI: 10.1037/h0042519.
- [2]D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning Representations by Back-Propagating Errors," *Nature*, vol. 323, pp. 533–536, 1986. DOI: 10.1038/323533a0.

- [3]Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. DOI: 10.1109/5.726791.
- [4]I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: <https://www.deeplearningbook.org/>.
- [5]C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [6]A. Geron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.
- [7]F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://www.jmlr.org/papers/v12/pedregosa11a.html>.

Appendix A: Implementation Summary

Dataset	Scikit-learn digits dataset - 1,797 samples of 8×8 grayscale images
Input Layer	64 neurons - one per pixel feature
Hidden Layer	32 neurons - Sigmoid activation
Output Layer	10 neurons - Softmax activation (digits 0–9)
Loss Function	Cross-entropy loss
Optimizer	Full-batch gradient descent via backpropagation
Learning Rate	0.5
Epochs	500
Train / Test Split	80% training (1,437 samples) / 20% testing (360 samples)
Label Encoding	One-hot encoding for output classes
Normalization	Pixel values scaled from 0–16 to 0–1
Final Test Accuracy	Approximately 94.44%

Table A1: Complete Implementation Parameters

Appendix B: Generated Results

The following figures were generated during the training and evaluation of the neural network:

- Figure 1 - Training loss curve: shows steady decrease from initial random weights toward a low-error state over 500 epochs.
- Figure 2 - Training and testing accuracy curves: both improve throughout training with a small and stable generalization gap.
- Figure 3 - Confusion matrix: 10×10 matrix comparing predicted digit classes against true labels; strong diagonal values confirm accurate classification.
- Figure 4 - Sample predictions: a grid of test images annotated with true labels (T) and predicted labels (P), illustrating both successes and failure cases.